Chapter 51 - One Effect, Six CPUs

The rotozoomer is a good comparison effect because the hardware job does not change. The VideoChip still owns Mode 7. The blitter still receives the same registers. The front buffer, texture, back buffer, and VBlank rhythm stay on the same bus.

What changes is the CPU idiom.

51.1 The Common Contract

Every version must do these jobs:

1. Enable VideoChip.
2. Place or build a texture.
3. Start its chosen audio engine.
4. Advance angle and scale state.
5. Convert table values into six Mode 7 parameters.
6. Establish RGBA32 pixels and compatible COPY state.
7. Start the blitter.
8. Wait for completion and present the frame.

That is the contract. A CPU chapter teaches how to execute instructions. This chapter teaches what remains the same after the instruction set changes.

51.2 CPU Comparison

CPU	Rotozoomer idiom	Main lesson
IE64	Wide native registers and fixed instruction width.	Keep many addresses and fixed-point values live at once.
IE32	Compact RISC-style code with explicit fixed-point helpers.	A small native CPU can still drive the same MMIO contract.
6502	Banked access, byte arithmetic, helper routines.	Use the hardware blitter because software pixels are not practical.
Z80	Register pairs, table copies, banked ranges.	Keep the bus contract visible through the adapter.
M68K	Orthogonal addressing and 68020-class integer work.	Use clean longword MMIO writes and table arithmetic.
x86	Flat 32-bit addressing and signed multiply.	Use familiar integer instructions against IE's bus.

The table is not a ranking. It is a reading guide.

51.3 The Same Blitter Writes

Each CPU eventually performs this sequence:

```
BLT_OP          = 5
BLT_SRC         = texture address
BLT_DST         = back buffer address
BLT_WIDTH       = 640
BLT_HEIGHT      = 480
BLT_SRC_STRIDE  = 1024
BLT_DST_STRIDE  = 2560
VIDEO_COLOR_MODE = 0
BLT_FLAGS       = 0
BLT_MODE7_U0    = computed U origin
BLT_MODE7_V0    = computed V origin
BLT_MODE7_DU... = computed deltas
BLT_CTRL        = 1
```

The 6502 may reach those registers through an adapter. The Z80 may use its own mapped view. M68K and x86 may use absolute long addresses. The device is still the same VideoChip.

The standalone versions begin from reset state, where the colour mode and blitter flags are already zero. The 6502 and Z80 sources also write those values explicitly so an earlier CLUT8 or raster-operation setting cannot leak into Mode 7. That explicit setup is the safer pattern when the effect is part of a larger programme.

51.4 Different Audio Choices

The shipped rotozoomers deliberately vary the sound engine. The point is the same as the video comparison: the audio engines are cards on the same machine.

CPU version	Example audio path
IE64	POKEY/SAP-style playback path.
IE32	AHX playback path.
6502	PSG/AY playback path.
Z80	SID-style path through the shared audio block.
M68K	TED playback path.
x86	PSG playback path.

The CPU does not have to mix samples itself. It starts an audio engine and returns to video work.

51.5 Reading A Port

When reading a port to another CPU, use this order:

1. Find the constants: framebuffer, texture, back buffer, stride.
2. Find the initialisation: video mode, framebuffer base, audio start.
3. Find the table lookups: angle, scale, sine, reciprocal.
4. Find the six Mode 7 parameters.
5. Find the blitter-completion wait and presentation step.
6. Find the accumulator advance.

Only after that should you study the CPU-specific tricks.

The IE64, IE32, 6502, Z80, M68K, and x86 versions render into an off-screen buffer, wait for completion, wait for a VBlank edge, and change `VIDEO_FB_BASE`. This is different from copying a back buffer into the displayed framebuffer. The VBlank edge is a presentation point, not a guarantee that every loop sustains `60` frames per second.

The 6502 and Z80 versions load their texture through the File I/O device. Their names are resolved beneath the guest File I/O root described in Chapter 35.

51.6 What The Comparison Proves

The six versions are not six separate machines. They are six views of one shared hardware contract. If you understand the BASIC version and the Mode 7 register block, you can read every port by asking one question:

```
How does this CPU compute and write the same six values?
```

Chapter 52 returns to BASIC and makes the source texture move before Mode 7 sees it.

51.7 The Version That Stops Its CPU

There is a seventh implementation with a different division of labour. IE64 runs only during setup. It clears the arithmetic state, builds `65,536` compact affine records, expands their bytes into a layout which the blitter can select, starts MIDI, enables Copper, and executes `HALT`.

The handover at the end of the bootstrap has this shape:

```
; Show the completed VideoChip framebuffer.
move.q  r2,#FRAMEBUFFER_BASE
store.l r2,VIDEO_FB_BASE(r0)
move.q  r2,#1
store.l r2,VIDEO_CTRL(r0)

; Start looping MIDI playback before IE64 stops.
move.q  r2,#MUSIC_BASE
store.l r2,MIDI_PLAY_PTR(r0)
move.q  r2,#MUSIC_LENGTH
store.l r2,MIDI_PLAY_LEN(r0)
move.q  r2,#5
store.l r2,MIDI_PLAY_CTRL(r0)

; Copper begins from this list once per presented frame.
move.q  r2,#COPPER_LIST_BASE
store.l r2,COPPER_PTR(r0)
move.q  r2,#3
store.l r2,COPPER_CTRL(r0)
halt
```

These are selected operations rather than a complete listing. `FRAMEBUFFER_BASE`, `MUSIC_BASE`, `MUSIC_LENGTH`, and `COPPER_LIST_BASE` stand for addresses and a length established earlier in the programme. The register values and lookup tables must already have been prepared before `HALT`. The important point is the order: prepare memory, start the independent player, enable the per-frame controller, then stop IE64.

After the handover, the work is divided like this:

Part	Work after `HALT`
IE64	None. Its retired-instruction count remains fixed.
Copper	Restarts once per presented frame, selects records, patches later operands, and starts blitter commands in order.
Blitter	Advances the bit-sliced angle and scale phases, selects the six affine values, and renders Mode 7.
MIDI player	Continues playback independently of IE64.
Presentation hold	Keeps the preceding completed picture visible until the next Mode 7 result is complete.

The angle and scale phases are stored as `16` bit planes. Each plane is all zeroes or all ones across the candidate set. Boolean blits therefore perform one stage of an addition or selection across every candidate at once. One selection chooses an angle table. A second selection chooses the scale entry within that table. Four byte lanes for each selected value then replace the operands of later Copper moves. Those moves write `U0`, `V0`, `DU_COL`, `DV_COL`, `DU_ROW`, and `DV_ROW` before starting Mode 7.

This is self-modifying Copper data, not self-modifying IE64 code. Copper instruction words remain in place; blitter copies change selected data words which later `MOVE` instructions consume.

When `rotozoomer_nocpu.ie64` is present on the current disk volume, run it from BASIC:

```
RUN "rotozoomer_nocpu.ie64"
```

The texture should continue rotating and zooming while MIDI plays. The programme performs a finite IE64 bootstrap first, so this is not a machine which starts without a CPU. It proves that, after setup, Copper, the blitter, VideoChip presentation and an audio player can sustain the effect without further CPU instructions.

The device status registers provide the corresponding checks. `COPPER_STATUS` reaches `HALTED` at the end of each list and returns to `RUNNING` on the next frame. `BLT_STATUS` must finish with `DONE` set and `ERR` clear. `MIDI_PLAY_STATUS` keeps `BUSY` set while the looping music plays. The changing picture is the final proof that later Copper passes are selecting different affine records.

The comparison question is now:

```
How can the custom chips select and submit the same six values after the
CPU has stopped?
```